ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

Faculty of Information Technology

WEB TECHNOLOGIES AND API SERVICES

FINAL PROJECT REPORT

Production Management System

With AI Edition

REST API Integration, Machine-Learning Models and Real-Time Prediction

Case study: Le Mieux Ltd

Course	Web Technologies and API Services
Lecturer	Kundan Kumar
Group	Group 13
Submission date	November 2026

Group Members

No.	Full name	Registration number
1	Iradukunda Muhire Bertin	20251MBI046
2	Isaac Nkusi	20251MBI020
3	Rwabukanga Jean de Dieu	20251MBI010

Abstract

Le Mieux Ltd is a leather manufacturer that produces shoes, belts, bags, and folders. In the midterm project, Group 13 built a Django-based Production Management System that brought production, inventory, and human-resources records together. That system described the present state of the factory but could not anticipate what would happen next. This final project extends it into a decision-support platform without changing any of its original business rules.

The extension adds a versioned REST API of 16 endpoints, a training dataset of 2,003 finished production batches, and two machine-learning models trained on data read through the API: a delay-risk classifier (ROC-AUC 0.934) and a duration regressor (mean absolute error 1.70 days). The models are deployed inside Django with a version registry, an in-memory inference service, and self-healing retraining. Every new order that arrives through the API, a simulator, or a web form is scored immediately, and the results appear on an enhanced dashboard together with the recommended artisan, a stock check, and plain-language advice. A public exchange-rate API is also consumed, with an offline fallback. The work is verified by 40 automated tests and a manual check under all three user roles.

Keywords: *Django, REST API, machine learning, gradient boosting, real-time prediction, production planning, dashboard*

Table of Contents

1. Introduction 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Aim and Objectives 1

1.4 Scope and Constraints 2

2. Background and Key Concepts 2

3. System Analysis and Design 3

3.1 The Existing System 3

3.2 Design Principles 3

3.3 Architecture 4

3.4 Technology Stack 4

3.5 Users, Roles and Permissions 5

4. Data Layer 5

4.1 New Data Models 5

4.2 Data Sources 6

4.3 Synthetic Data Generation 6

4.4 Dataset Overview 7

5. REST API 7

5.1 Design Principles 7

5.2 Authentication and Authorisation 7

5.3 Endpoint Reference 8

5.4 Example: Ingesting an Incoming Order 9

5.5 Open API Integration (Exchange Rates) 9

6. Machine Learning 9

6.1 Problem Formulation 9

6.2 Data Acquisition through the API 10

6.3 Feature Engineering and Leakage 10

6.4 Evaluation Method 10

6.5 Algorithm Comparison 11

6.6 Results of the Deployed Models 11

6.7 What the Models Learned 12
6.8 Limitations of the Models 12
7. Deployment in Django 13
7.1 Model Registry and Inference Service 13
7.2 Self-Healing, Continuous Learning and Retraining 13
8. Real-Time Prediction of Incoming Data 14
8.1 The Ingestion Pipeline 14
8.2 Risk Levels, Artisan Choice and Recommendations 14
8.3 From Prediction to Action 15
9. User Interface and Dashboard 16
9.1 Enhanced Dashboard 16
9.2 AI Insights and Order Detail 16
9.3 What-if Predictor and Model Center 17
9.4 Task Risk Card, Alerts and Dark Mode 19
10. Testing and Quality Assurance 19
11. Integration with the Existing System 20
12. Security Considerations 20
13. Challenges and Lessons Learned 21
14. Limitations and Future Work 21
15. Conclusion 22
References 23
Appendix A: Installation and Operation Guide 24
Appendix B: Glossary 25

1. Introduction

1.1 Background

Le Mieux Ltd is a manufacturer of handcrafted leather goods. It converts raw materials such as leather, thread, polish, buckles and zippers into shoes, belts, bags and folders. Every product follows a recipe of materials, every production order is carried out by an artisan, and every artisan has a contract and may request leave.

In the midterm project, Group 13 replaced the company's paper records and separate spreadsheets with the Production Management System (version 1.0), a Django web application built with the AdminLTE template. It is organised in three Django apps (inventory, production and employees) with seven related models. It predicts the production cost and profit margin of each product from its Bill of Materials (BOM), refuses a production task when the warehouse holds too little material, deducts and refunds stock automatically, prevents artisans on approved leave from being assigned work, and enforces a strict status workflow for every task.

1.2 Problem Statement

Version 1.0 answers the question "what is the situation now?" but not "what is likely to happen next?". In daily operation, the production manager still has to guess whether a new order can realistically be delivered by the date the customer wants, which artisan should take a job so that it is finished on time, which open tasks are drifting towards a delay before they become overdue, and which raw materials will run short if all pending orders are accepted. In addition, other systems such as a sales channel or a workshop terminal have no way to exchange data with the ERP automatically.

1.3 Aim and Objectives

The final project aims to turn the system from a record-keeping tool into a decision-support tool by adding data exchange through an API and machine learning. The lecturer set five requirements, which the group adopted as its objectives:

1. Read data from an API: an Open API, a registered API, or an API created with Django.
2. Train a model using the original data from the API or synthetic data.
3. Deploy the trained model inside Django.

4. Let the system read new incoming data, make predictions on it and display them on an improved dashboard.
5. Add all of this without breaking the way the current system works.

Table 1 shows how each requirement was fulfilled and which chapter describes it.

Table 1: *Requirements and how they were fulfilled*

#	Requirement	How it was fulfilled	Chapter
1	Read data from an API	Own REST API at /api/v1/. The training pipeline reads GET /api/v1/records/ in-process or over HTTP. An Open API (exchange rates) is also consumed.	4, 5, 6
2	Train the model	2,000 synthetic batches generated from the real catalogue, plus the real completed tasks. Two models, with three algorithms compared for each.	4, 6
3	Deploy in Django	Model registry, joblib artefacts, cached inference service, self-healing and retraining from the web page, the command line or the API.	7
4	New data, prediction, dashboard	POST /api/v1/orders/ scores an order the moment it arrives. Order simulator, live feed refreshing every 15 seconds, new KPIs, risk monitor and charts.	8, 9
5	Do not break the system	Additive design (new app and new tables only), fail-safe dashboard hook, and regression tests of every original business rule.	10, 11

1.4 Scope and Constraints

The extension covers production-planning decisions: delivery risk, duration, artisan assignment, and material demand. It does not change the HR, costing, or inventory rules of version 1.0, nor does it replace the manager's judgment: every prediction is presented with its reasons, and nothing is created automatically without a manager's confirmation. Two constraints shaped the design. The factory has only five artisans and very few completed tasks, which is far too little real history for machine learning, so a faithful synthetic history had to be generated. The ERP must keep working when the internet is unavailable, so no feature may depend on an external service.

2. Background and Key Concepts

REST APIs. A REST (Representational State Transfer) API lets programs exchange data with a web application over HTTP, using verbs such as GET to read data and POST to send data (Fielding, 2000). The data travels as JSON, a text format that every programming language can read. A well-designed API is versioned, answers with consistent messages and status codes, and checks who is calling. In this project, the API is both the doorway through which other systems send orders to the ERP and the source from which the machine-learning pipeline reads its training data.

Machine learning for production planning. Supervised machine learning learns from past examples whose outcome is already known. *Classification* predicts a category, here "late" or "on time", together with a probability. *Regression* predicts a number, here the number of days a job will take. To estimate how a model will behave in practice, it is trained on older records and tested on newer records it has never seen. The best algorithm in this project is *gradient boosting* (Friedman, 2001), which builds many small decision trees one after another, each correcting the mistakes of the previous ones. It was compared with a linear model and a random forest (Breiman, 2001), all implemented with scikit-learn (Pedregosa et al., 2011).

Deploying a model in a web framework. A trained model is useful only when the application can reach it. The usual approach is to save the fitted model to a file (an artefact), record its version and quality in the database, load it once into memory, and call it from the views and the API whenever new data arrives. Three concerns follow: tracking versions, making sure a model saved on one computer still loads on another, and explaining each prediction to non-technical users. Chapters 6 to 8 show how the project addresses them.

3. System Analysis and Design

3.1 The Existing System

The extension builds on the three apps of version 1.0: inventory (raw materials, products and Bills of Materials), production (tasks, the status workflow, stock control and the dashboard) and employees (artisans, contracts and leave requests). These apps were not restructured; all new work lives in a fourth app called analytics.

3.2 Design Principles

- **Additive.** New code and new tables only. No original field, URL or business rule was removed.
- **One business logic for every channel.** The API calls the same service functions as the web pages, so an order sent by a script and an order typed in a form are validated and scored identically.
- **Fail-safe.** An error in the analytics layer can never block the dashboard or the saving of a task, and the external exchange-rate service has an offline fallback.
- **Explainable.** Each prediction carries its reasons, recommendations, model version, and the exact inputs given to the model.
- **The manager stays in control.** Predictions advise; a person confirms before an order becomes a task.

3.3 Architecture

Figure 1 shows the architecture. The analytics app has three layers: the *API layer* (authentication, serialisation and endpoints), the *ML layer* (features, synthetic data, training, inference and descriptive analytics) and an *integration layer* (a signal, a context processor and template tags) that connects the AI to the existing pages without rewriting them. Three kinds of clients use the system: people through the web browser, machine clients such as the order simulator or a workshop terminal through the REST API, and administrators through manage.py commands such as train_models and simulate_orders.

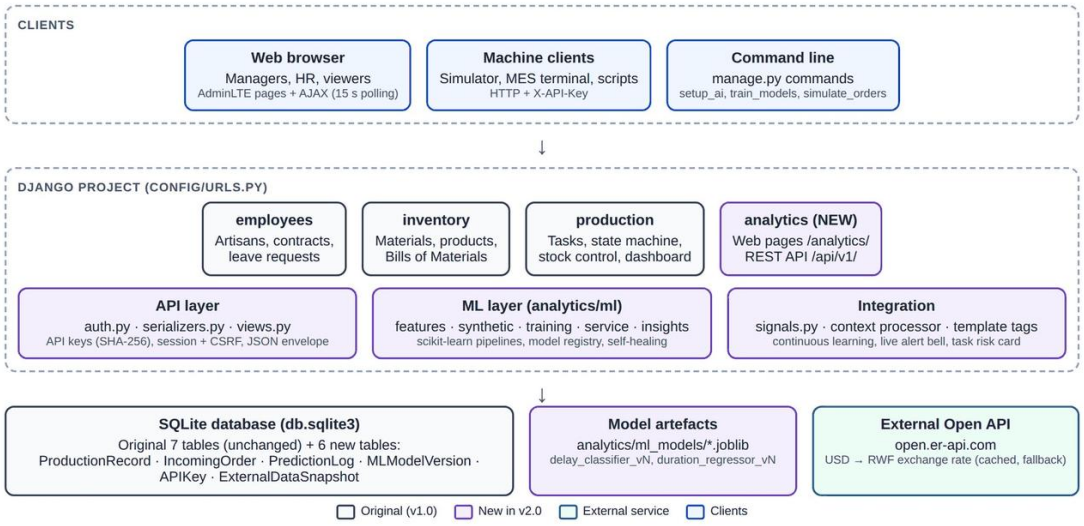


Figure 1: *System architecture of the AI Edition (purple = new components)*

3.4 Technology Stack

Table 2: *Technologies used*

Layer	Technology	Role
Back end	Python 3.12+, Django 6.1	Web framework, ORM, authentication, admin
Machine learning	scikit-learn ≥ 1.6, NumPy, SciPy, joblib	Model training, evaluation and persistence
Front end	Bootstrap 4, AdminLTE 3, Font Awesome, jQuery	Responsive layout and components (unchanged from v1.0)
Charts and tables	Chart.js 2.9, DataTables	Dashboard charts, sortable and searchable tables
Database	SQLite	Same database file as v1.0, with 6 new tables
External data	ExchangeRate-API open endpoint	Live USD → RWF rate (no key required)

3.5 Users, Roles and Permissions

The extension reuses the three roles of version 1.0. Table 3 summarises what each role can do with the new features; the original permissions are unchanged.

Table 3: *Permissions of the new features by role*

Capability	Viewer	Manager	Admin
View dashboard, AI Insights, Model Center, History, and API documentation	Yes	Yes	Yes
Run What-if predictions; read API endpoints	Yes	Yes	Yes
Submit or simulate incoming orders; convert or dismiss them	No	Yes	Yes
Retrain the models; create and revoke API keys	No	Yes	Yes
Django Admin for the new models	No	If granted	Yes

4. Data Layer

4.1 New Data Models

Six models were added to the database (Table 4). None of them change an existing table. Every link to an original model uses SET_NULL on deletion, so deleting a product, an artisan, or a task can never fail because of analytics data, and history is preserved through snapshot fields.

Table 4: *New data models*

Model	Purpose	Key fields
ProductionRecord	One finished batch: the training data	Snapshot of product, artisan and stock coverage; start, due and end dates; actual days; delayed; defective units; source
IncomingOrder	A new production request and its prediction	Reference, customer, channel, product, quantity, due date, delay probability, risk level, predicted days, recommended artisan, stock check, status
PredictionLog	Audit trail of every prediction served	Kind, inputs, outputs, model version, user, time
MLModelVersion	Registry of trained models	Name, version, algorithm, file, metrics, candidates, feature importances, training and test sizes, active flag
APIKey	Credentials for machine clients	Name, owner, prefix, SHA-256 hash, active flag, last used
ExternalDataSnapshot	Last good value from the Open API	Key, payload, time fetched

4.2 Data Sources

The models learn from the ProductionRecord table, which is fed by three sources: the **synthetic history** of 2,000 batches described in Section 4.3; **live records**, because every production task that reaches the status Completed is copied automatically into the history, with its real duration and outcome, by a Django post_save signal (3 records at present); and **external pushes**, since another system can send a finished batch with POST /api/v1/records/.

4.3 Synthetic Data Generation

The live database holds only a handful of tasks, so a synthetic history was generated. It stays faithful to the company: every synthetic batch uses a real product with its real Bill of Materials, and a real artisan with his or her real department and contract. Only the dates, quantities, and outcomes are simulated.

Each batch first receives an *engineering standard duration*, the time a typical artisan would need under normal conditions. It depends on the product category (shoes take longest per unit, followed by bags, belts, folders and accessories), the number of materials in the recipe, the quantity of material and the batch size. The *actual duration* is then obtained by applying the chain of factors in Table 5, each of which a production manager would recognise. The time allowed by the manager is the standard duration multiplied by a random generosity factor, and a batch counts as delayed when its actual duration exceeds the time allowed.

Table 5: *Factors that shape the actual duration of a synthetic batch*

Factor	**Effect on the duration**
Artisan skill	A hidden speed factor per artisan, from 25 % faster to 20 % slower than standard
Contract type	Permanent artisans 5 % faster, temporary 5 % slower, interns 20 % slower, no contract 10 % slower
Workload	Each other active task of the artisan lengthens the job by about 8 %
Speciality	18 % longer when the artisan works outside his or her department
Peak season	15 % longer when the batch starts in November or December
Stock shortage	A restock wait of roughly 2 to 15 days, longer when the shortage is bigger
Random variation	A natural spread so that no two batches behave identically

The dataset covers 2,000 batches over the last 540 days (about 18 months), generated with a fixed random seed (42) so that it is reproducible. 55 % of the batches are given to an artisan from the department matching the product category, 12 % start with a shortage of at least one material, and about 1.5 % of units are defective. Together with the 3 real tasks, the history holds 2,003 batches, of which 34.4 % (690) finished late. Because the rules that generate the

data are written down, the team can check that the models learned the right things; Chapter 6 shows that they did.

4.4 Dataset Overview

Figure 2 shows the Production History page, where the training data can be browsed, filtered by source and searched. The late rate differs by category: shoes 41.3 %, accessories 39.0 %, belts 34.7 %, bags 31.8 % and folders 26.2 %, which matches the longer and more complex production of shoes.

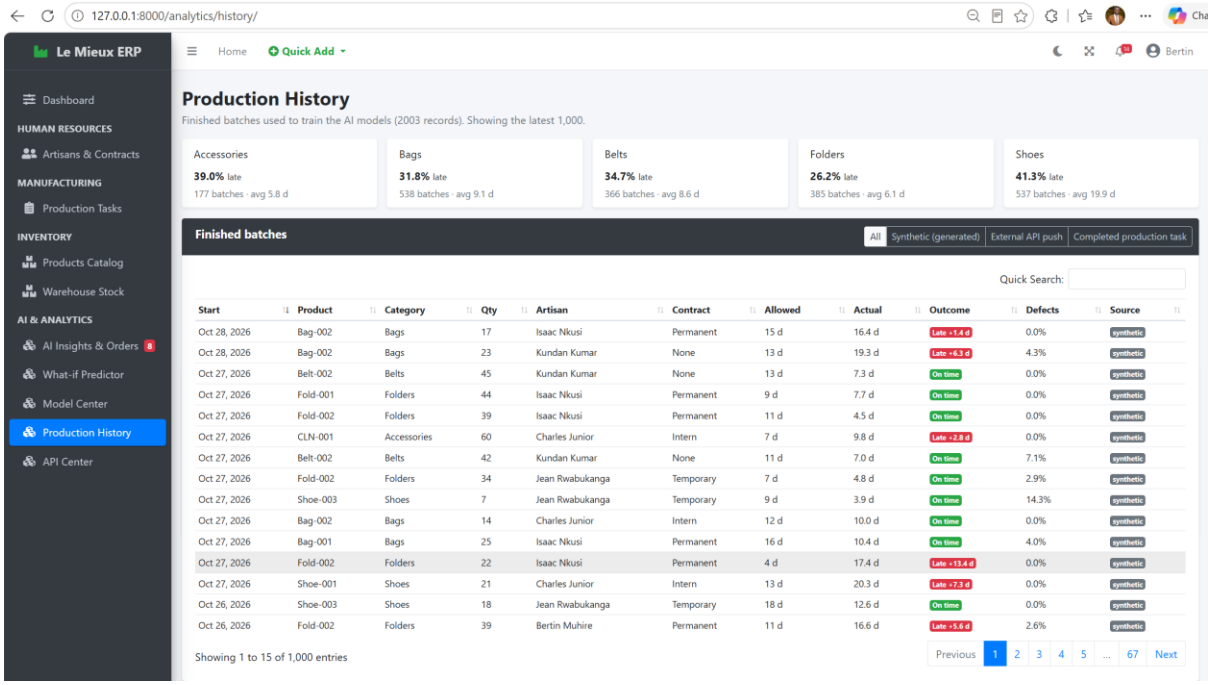


Figure 2: *Production History page: the training data with the delay rate per category*

5. REST API

5.1 Design Principles

- **Versioned base path** /api/v1/, so that future changes do not break existing clients.
- **JSON everywhere**, including errors, with one response envelope: status, data and meta on success, status and error on failure.
- **Correct HTTP status codes**: 200, 201, 400, 401, 403, 404, 405, 409 and 503.
- **Same business logic as the web pages**, and **no sensitive data**: salaries are deliberately excluded from the employee's endpoint.

5.2 Authentication and Authorisation

Two authentication methods are accepted, checked in this order. The first is an *API key* sent in the X-API-Key header. Keys are random 43-character strings prefixed with lm_. Only a SHA-256 hash is stored, so a stolen database does not reveal usable keys; the plain key is shown once, at creation, and a key acts with the permissions of its owner and can be revoked at any time. The second is the *browser session* of a logged-in user, used by the dashboard's AJAX calls; for POST requests, the CSRF token is verified manually, so session-based writes keep Django's CSRF protection.

Every GET endpoint needs an authenticated user, except the public index and health endpoints, and every POST that changes data needs a staff user. The only exception is POST /predict/, which stores nothing and may be called by any authenticated user.

5.3 Endpoint Reference

The API has 16 endpoints, which offer 18 operations (Table 6). The API Center page of the application documents each of them with examples and lets managers create and revoke API keys.

Table 6: *REST API endpoints*

Method	Path	Description	Access
GET	/api/v1/	API index with the list of endpoints	public
GET	/api/v1/health/	Service status and active model versions	public
GET	/api/v1/products/	Products with BOM, cost and margin	read
GET	/api/v1/materials/	Raw materials, stock value, low-stock flag	read
GET	/api/v1/employees/	Artisans with department and contract (no salary)	read
GET	/api/v1/tasks/	Production tasks with progress and overdue flag	read
GET	/api/v1/tasks/<id>/prediction/	AI assessment of one task	read
GET	/api/v1/records/	Training data, paginated	read
POST	/api/v1/records/	Push a finished batch from an external system	staff

GET	/api/v1/orders/	Incoming orders with predictions	read
POST	/api/v1/orders/	Ingest a new order; it is scored instantly	staff
GET	/api/v1/orders/<id>/	One order with its full prediction	read
POST	/api/v1/predict/	Stateless what-if prediction (nothing stored)	read
GET	/api/v1/model/	Active models, metrics and version history	read
POST	/api/v1/model/retrain/	Retrain and redeploy both models	staff
GET	/api/v1/dashboard/summary/	KPIs and chart series	read
GET	/api/v1/forecast/materials/	Material demand forecast	read
GET	/api/v1/external/fx/	Open API exchange rate and inventory value in USD	read

5.4 Example: Ingesting an Incoming Order

The request below sends a new order. The reply, shortened here, contains the reference given to the order and its prediction. A request with mistakes returns every problem at once, with status 400 and a validation_error code that names each faulty field.

```
curl -X POST http://127.0.0.1:8000/api/v1/orders/ \
  -H "X-API-Key: lm_xxxxxxxx" -H "Content-Type: application/json" \
  -d '{"product_code": "Bag-002", "quantity": 5,
       "due_date": "2026-11-20", "customer": "Kigali Heights Boutique"}'

HTTP/1.1 201 Created
{"status": "ok", "data": {"reference": "ORD-261029-0009",
  "prediction": {"delay_probability": 0.0329, "risk_level": "Low",
    "predicted_days": 4.1, "recommended_artisan": "Bertin Muhire"}}}
```

5.5 Open API Integration (Exchange Rates)

The ERP also consumes the ExchangeRate-API open endpoint, which needs no key. The USD to RWF rate is used to show the value of the inventory in US dollars on the dashboard, which

is useful for export pricing. Because an ERP must never stop working when the internet is down, the client follows a four-step policy: serve the cached value if it is less than 6 hours old; otherwise call the Open API with a 5-second timeout; if the call fails, serve the last good value stored in the database; and if there has never been a good value, serve a fallback rate from the settings (1,450 RWF), clearly labelled "offline". The dashboard loads the rate asynchronously, so a slow network never delays the page, and the setting AI_EXTERNAL_APIS_ENABLED = False switches the call off completely.

6. Machine Learning

6.1 Problem Formulation

Two complementary questions are answered for every job, which is defined by a product, a quantity, a due date, and an artisan (Table 7).

Table 7: *The two prediction tasks*

Model	**Type**	**Target**	**Business question**
Delay-risk classifier	Binary classification	Whether the batch finishes after its due date	How likely is this job to be late?
Duration regressor	Regression	Actual number of days taken	How many days will this job really take, and when will it finish?

6.2 Data Acquisition through the API

To satisfy the requirement that the model is trained on data read from an API, the training pipeline never queries the training table directly. It requests GET /api/v1/records/ page by page (1,000 records per page) and works only with the JSON it receives. In the default *in-process* mode, the request goes through Django's complete request and response stack (routing, middleware, authentication, and serialisation) using an internal read-only service account that cannot log in, so no running server is needed. In *HTTP* mode, train_models --api-url ... --api-key ... reads from a running server exactly as an external client would. This mode was tested while the server was serving users, and the server picked up the new model version without a restart.

6.3 Feature Engineering and Leakage

Eighteen features are built by a single module that is used both by the training pipeline and by live inference, which guarantees that the model sees in production the same kind of input it saw during training (Table 8).

Table 8: *The 18 model features by group*

Group	Features	Meaning
Product (5)	Category, unit price, material cost per unit, number of materials, material quantity per unit	How complex and valuable the product is
Job (5)	Quantity, standard days, allotted days, slack ratio, required rate	Size of the job and how generous the plan is
Artisan (5)	Contract type, department match, workload, historical on-time rate, historical speed	Who does the work and how reliable he or she has been
Stock and time (3)	Stock coverage, start month, peak-season flag	Whether materials are available and the time of year

The two artisan-history features could easily leak the answer: if a batch's own outcome were included in the artisan's on-time rate, the model would look excellent in testing and fail in practice. The pipeline therefore computes them only from batches that finished before the current batch started, and requires at least three earlier batches; otherwise neutral defaults are used. An automated test checks this rule.

6.4 Evaluation Method

A random split would let the model learn from the future. The data is therefore split by time: the oldest 80 % of the batches (1,602, up to 18 July 2026) are used for training and the newest 20 % (401) for testing. This reproduces how the model is used in reality, trained on the past and applied to the future. After selection, the winning algorithm is refitted on all the records before deployment.

6.5 Algorithm Comparison

Three algorithms of increasing flexibility were compared for each task. The classifier is selected by ROC-AUC, which measures how well it ranks risky jobs above safe ones, and the

regressor by mean absolute error (MAE), which is expressed in days and is easy to explain. Gradient boosting won both comparisons (Tables 9 and 10, Figure 3).

Table 9: *Delay-risk classifier: comparison on the test set*

Classifier	**Accuracy**	**F1**	**ROC-AUC**
Logistic Regression (balanced)	0.778	0.683	0.852
Random Forest (250 trees)	0.838	0.764	0.925
Gradient Boosting (250 stages), selected	**0.855**	**0.779**	**0.934**

Table 10: *Duration regressor: comparison on the test set*

Regressor	**MAE (days)**	**RMSE (days)**	**R^2**
Ridge Regression	2.727	3.736	0.807
Random Forest (250 trees)	1.754	2.810	0.891
Gradient Boosting (300 stages), selected	**1.698**	**2.783**	**0.893**

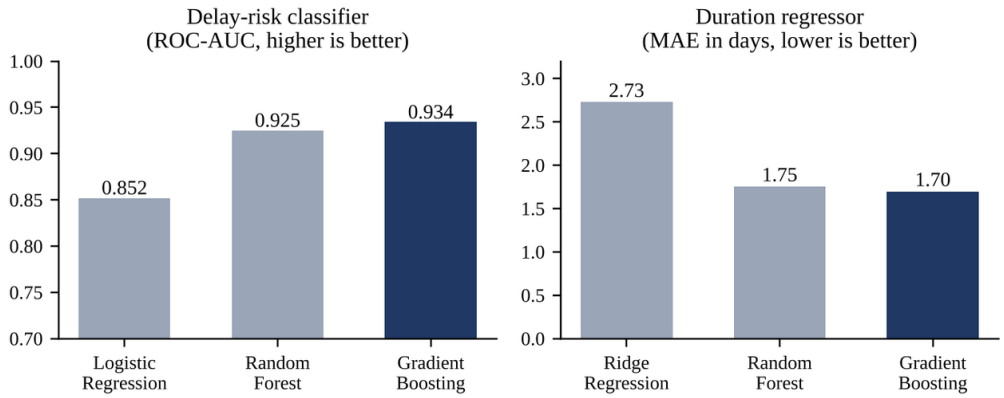


Figure 3: *Comparison of the three algorithms for each model*

6.6 Results of the Deployed Models

On the 401 unseen batches the delay-risk classifier reaches an accuracy of 85.5 %, a precision of 76.7 %, a recall of 79.1 % and a ROC-AUC of 0.934. In practice, when the system says “late”, it is right about three times out of four, and it catches about four out of five of the jobs that really end late (Table 11).

Table 11: *Confusion matrix of the classifier (test set, threshold 0.5)*

	Predicted on time	**Predicted late**
Actual on time (272)	241 correct	31 false alarms
Actual late (129)	27 missed	102 caught

The duration regressor has a mean absolute error of 1.70 days, an R^2 of 0.893 and a median relative error of 12.1 %, and 68.8 % of its predictions fall within ±20 % of the real duration. For batches that typically last 3 to 25 days, this is precise enough to promise realistic dates to customers.

6.7 What the Models Learned

The feature importance in Figure 4 shows that the models rely on meaningful factors. For delay risk, the most important are stock coverage (38.8 %, because a shortage means waiting for a restock), the slack ratio (23.9 %, how generous the plan is), the artisan's workload (8.9 %), and his or her historical on-time rate (7.7 %) and speed (5.2 %). For duration, the engineering standard dominates (78.1 %), followed by stock coverage (11.1 %) and workload (2.9 %). These are the mechanisms built into the data, which confirms that the models learned the causes of delay rather than accidental patterns.

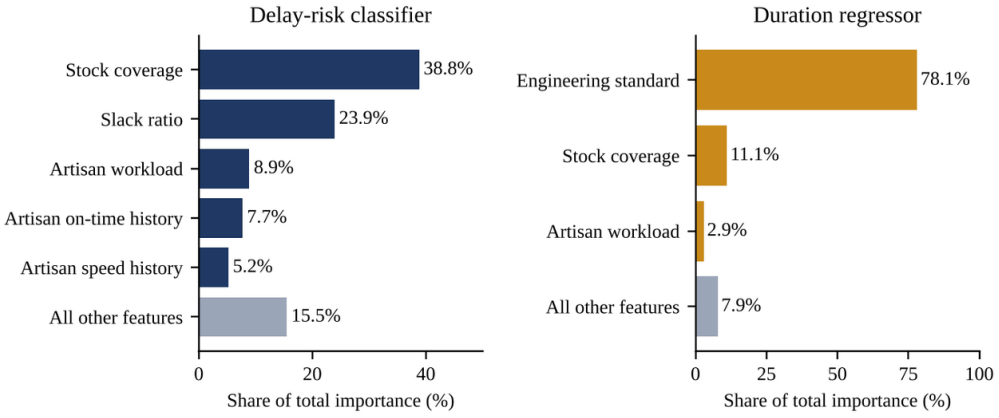


Figure 4: *Most important features of each model*

6.8 Limitations of the Models

- The models are trained mostly on synthetic data, so they reproduce the assumptions of Section 4.3 rather than the real behaviour of the factory. The continuous-learning loop (Section 7.2) corrects this progressively as real tasks are completed.

- The factory has five artisans, so predictions for a newly hired artisan rely on contract and department until three batches have been recorded for him or her.
- The probabilities are not formally calibrated. They are suitable for ranking and for the Low, Medium, and High bands, but should not be read as exact frequencies.

7. Deployment in Django

7.1 Model Registry and Inference Service

Each training run produces a new version of each model. The fitted pipeline is saved with joblib together with the scikit-learn version used, and a row in the MLModelVersion table records the algorithm, the metrics, the comparison of all candidates, the feature importances, the sizes of the training and test sets and who trained it. Only one version per model is active; older versions remain in the registry as history. Figure 5 shows the whole life cycle, from data to live predictions and back.

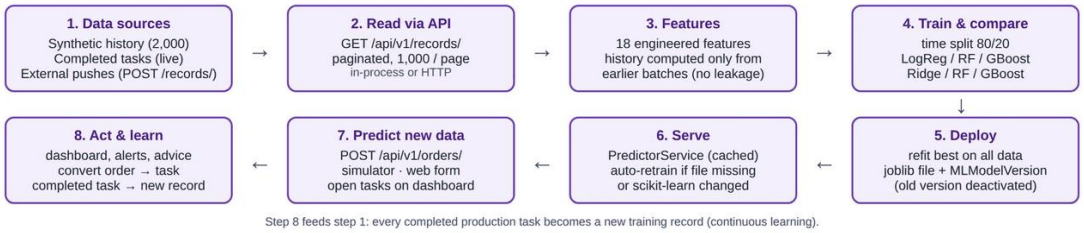


Figure 5: *Machine-learning life cycle, from data to live predictions and back*

The PredictorService is the single-entry point for predictions. It loads the active versions once and keeps them in memory, protected by a lock so that several requests can be served safely. When a new version is activated, even by a training run started from another terminal, the service notices the change on the next prediction and loads it without restarting the server. The predictions for all candidate artisans are computed in one batch call, which keeps the scoring of an order fast.

7.2 Self-Healing, Continuous Learning and Retraining

Machine-learning files are fragile when a project is moved between computers: a model saved with one scikit-learn version may not load with another. The service therefore checks every model it loads. If the file is missing, cannot be read, or was produced by a different scikit-learn version, it retrains automatically from the data in the API (about 10 seconds) and continues; if

the database holds no training data at all, it first generates the synthetic history. A fresh installation therefore works on the first click. Both situations are covered by automated tests.

When a manager marks a production task as Completed, a signal handler creates a ProductionRecord with the real start date, end date, duration and outcome. The handler runs only once per task and never blocks the save: any error is logged and ignored, so analytics can never break the core workflow. The next retraining includes these real records. Retraining can be started with the Retrain models now button of the Model Center, with python manage.py train_models, or with POST /api/v1/model/retrain/; a lock prevents parallel runs and a run takes about 8 seconds.

8. Real-Time Prediction of Incoming Data

8.1 The Ingestion Pipeline

New data enters the system as *incoming orders*: production requests from the storefront, corporate clients, the online shop or export partners. An order can arrive through the API, through the order simulator, or through the What-if Predictor, and in every case the same function is used, so the result does not depend on the channel. Figure 6 shows its six steps: authenticate and authorise, validate, build the context (recipe, stock, artisans on duty, workloads and history), predict for every candidate artisan in one batch call, interpret the result, and save it and respond. The dashboard’s live feed shows the new row within 15 seconds.

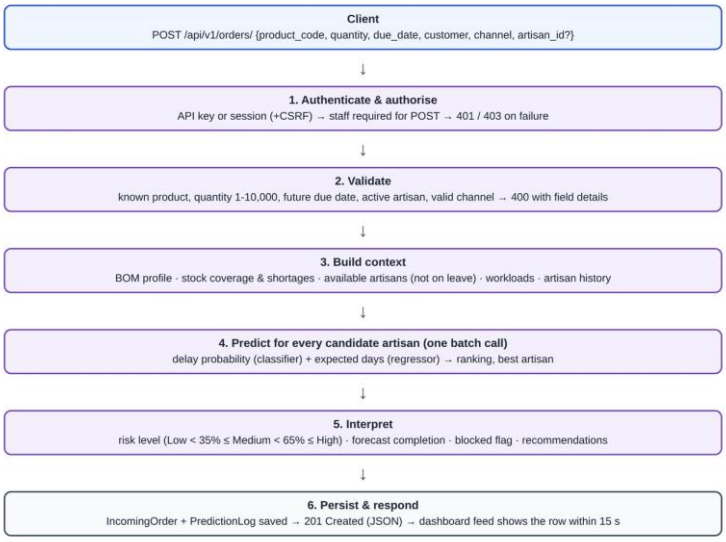


Figure 6: *Real-time ingestion and scoring of an incoming order*

8.2 Risk Levels, Artisan Choice and Recommendations

The delay probability is translated into a risk level that a manager can act on (Table 12).

Table 12: *Risk levels*

Risk level	**Delay probability**	**Meaning for the manager**
Low	below 35 %	The plan is realistic; standard scheduling is enough.
Medium	35 % to 65 %	Watch the job; consider a more suitable artisan or a little more time.
High	65 % and above	The job is likely to be late; act before accepting it (reassign, extend, restock or split).
Overdue	not applicable	An open task already past its due date; no prediction is needed.

For each order the system lists the active artisans who are not on approved leave during the job, which is the same rule the original system enforces, computes the delay probability and expected duration for each of them, and ranks them from the safest to the riskiest. The first becomes the recommended artisan. The Bill of Materials is also compared with the warehouse: when stock is insufficient the job is marked *Blocked until restocked*, because the original stock rule would refuse to create the task, and the missing quantity of each material is listed. The system then writes plain-language recommendations: restock a short material; assign the job to another artisan when this lowers the risk by more than 10 points; rebalance work when the chosen artisan already has three or more active tasks; allow about 10 % more days or split the batch when the expected duration exceeds the time allowed; otherwise confirm that the plan is realistic.

8.3 From Prediction to Action

The same models assess every open production task, on the dashboard, on the AI Insights page, on each task's detail page and through GET /api/v1/tasks/<id>/prediction/. For an existing task the materials were already reserved, so stock is not treated as a risk; overdue tasks are flagged directly, and for tasks in progress the finishing date is projected from the actual production pace.

A manager can turn an order into a real production task with one click. The standard task form opens pre-filled with the product, quantity, due date, the recommended artisan and a summary of the prediction. Saving it goes through all the original validation rules (stock, leave, status workflow and due date); if they pass, the materials are deducted exactly as before and the order is marked Converted. Orders that are not accepted can be dismissed. To demonstrate real-time behaviour, python manage.py simulate_orders generates realistic orders and sends them one by one, either in-process or over HTTP with --api-url and --api-key like an external system; managers can also press the Simulate orders button in the application.

9. User Interface and Dashboard

All new pages use the same AdminLTE layout, colours and components as version 1.0, work in light and dark mode, and are reached from a new sidebar section named AI & Analytics. Every list has a clear empty state, and every AI widget shows a short explanation, instead of an error page, when the AI engine is unavailable.

9.1 Enhanced Dashboard

The original dashboard is kept in full: the alert banner, the four summary boxes, the recent tasks and the active workload. A new row of KPIs was added under the summary boxes: inventory value in RWF and USD, on-time rate (overall and for the last 90 days), high-risk open tasks and AI model quality (Figure 7). Below it, an AI Production Intelligence section adds the Delay-Risk Monitor (open tasks ranked by risk, with the forecast completion date), a task-status chart, the Live Incoming Orders feed, the Material Demand Forecast, the monthly output and delay rate over 12 months, and the delay rate by category.

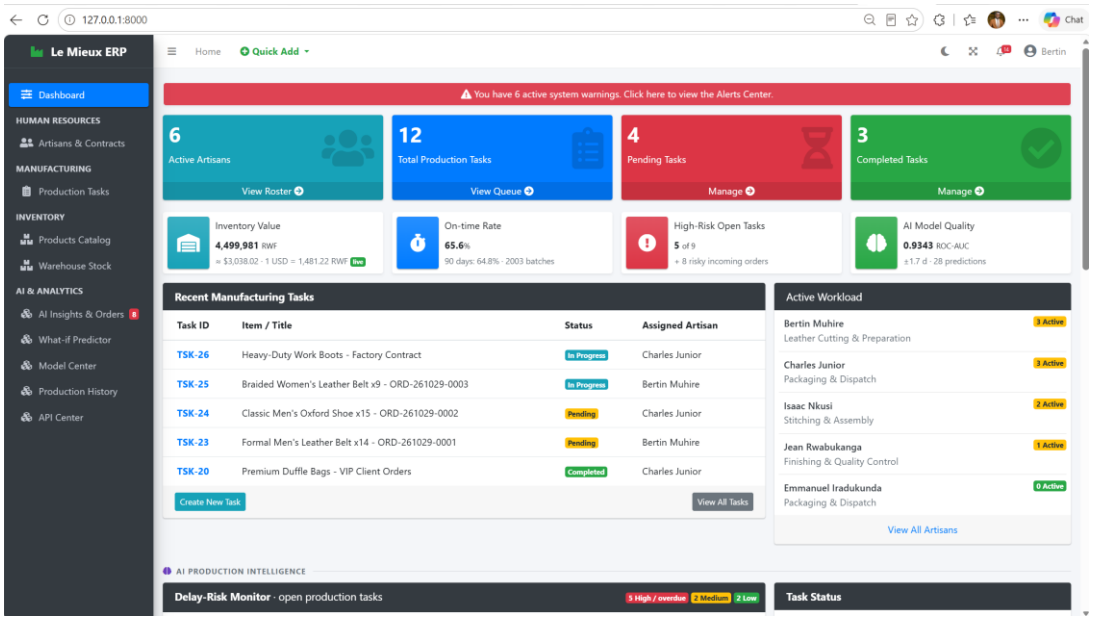


Figure 7: *Dashboard: original widgets, the new KPI row and the Delay-Risk Monitor*

9.2 AI Insights and Order Detail

The AI Insights page is the inbox of incoming orders (Figure 8). Counters show new, high-risk, converted and total orders, and the list can be filtered by status and risk. Each order shows its risk with a bar, the expected duration and finish date, the requested or recommended artisan and the stock status. Managers can view, convert or dismiss each order and simulate new ones.

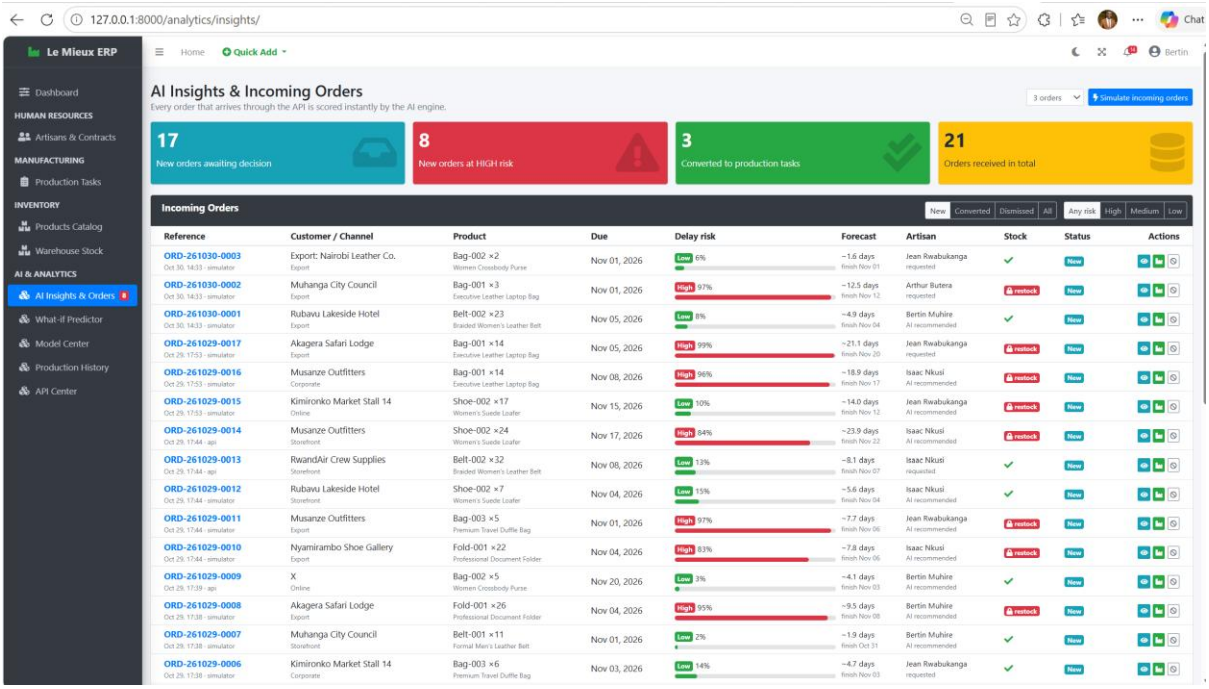


Figure 8: *AI Insights & Orders page*

The order page shows the prediction made at arrival (Figure 9): a risk gauge, the expected duration, the time allowed, the forecast completion, the recommendations, the stock check with the missing quantities, the ranking of all artisans and, for transparency, the exact feature vector given to the model.

Figure 9: *Order detail with the prediction made at arrival (a job blocked by a stock shortage)*

9.3 What-if Predictor and Model Center

Managers and viewers can test a job before committing to it in the What-if Predictor (Figure 10): they choose a product, a quantity, and a due date, optionally an artisan, and click Predict outcome. Nothing is stored except an audit-log entry, and a manager can then submit the same job as an incoming order.

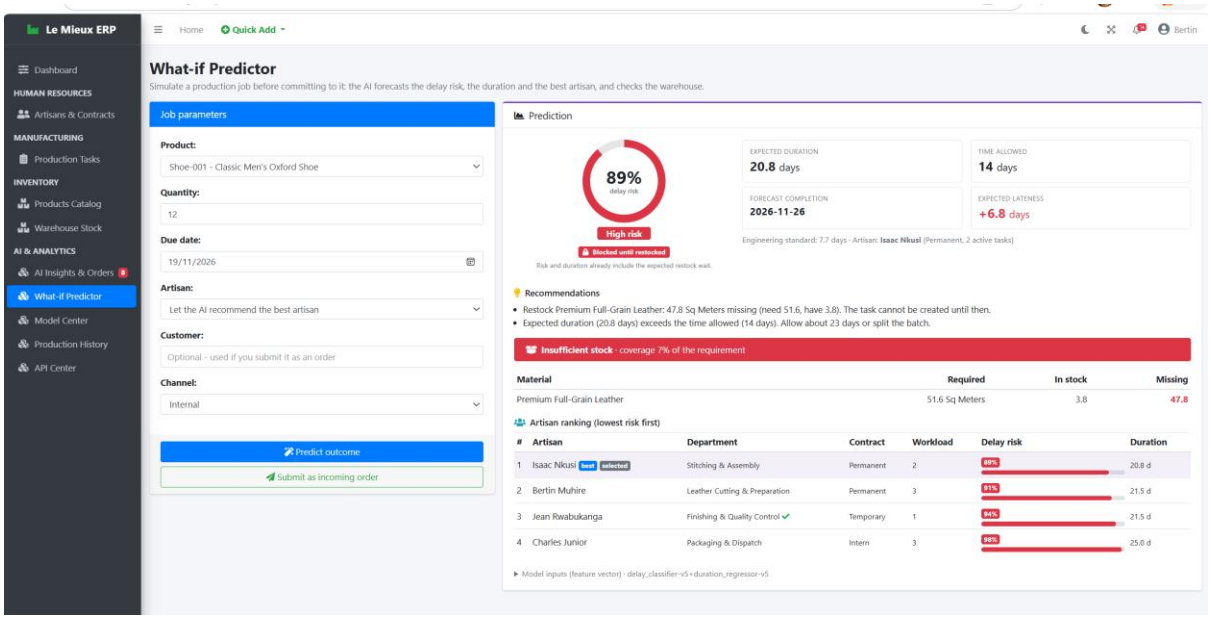


Figure 10: *What-if Predictor*

The Model Center explains the AI to managers (Figure 11): the pipeline, the metrics of each active model on unseen data, the confusion matrix, the algorithm comparison, the feature importance, the training data by source, the version registry, and the audit log of recent predictions. Managers can retrain the models with one click.

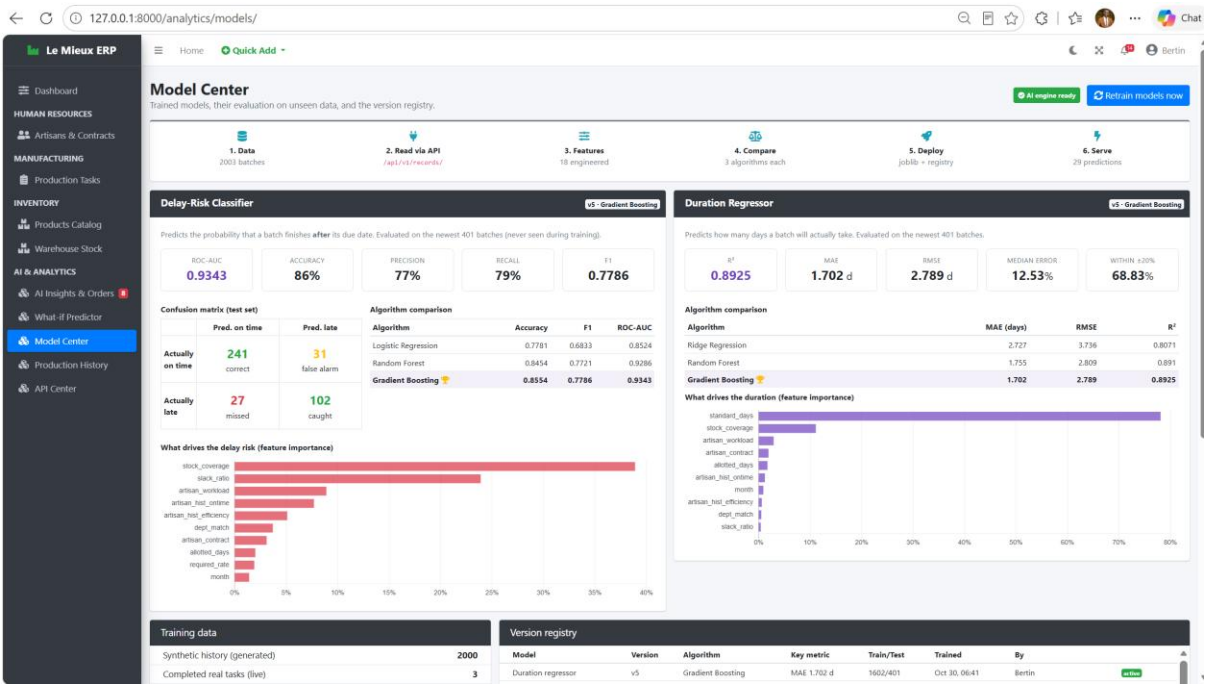


Figure 11: *Model Center: metrics, confusion matrix, algorithm comparison and feature importance*

9.4 Task Risk Card, Alerts and Dark Mode

Every production task page now has an AI Risk Assessment card with the delay probability, the expected duration, the days allowed, the forecast finish, and the recommendations; for a completed task, it explains that the real outcome has been added to the training data. The notification bell shows the real number of alerts (overdue tasks, low-stock materials and high-risk incoming orders) with a breakdown, and the sidebar shows a red badge next to AI Insights & Orders when high-risk orders are waiting. The existing dark mode applies to all the new widgets and charts.

10. Testing and Quality Assurance

The file analytics/tests.py contains 40 automated tests. They build a small factory (artisans, contracts, materials, products, recipes, and the three user roles), generate 400 synthetic records, and train real models in a temporary folder. They run with python manage.py test in about 20 seconds, and all 40 pass (Table 13).

Table 13: *Automated test suite (40 tests, all passing)*

Group	Tests	What is verified
Machine-learning pipeline	9	Realistic synthetic data; training through the API; new version on retraining; quality thresholds; no target leakage; stock-shortage detection; artisans on leave excluded; self-healing
REST API	14	Authentication (no key, wrong key, revoked key); every read endpoint; salaries not exposed; pagination; ingestion and scoring; viewer permissions; validation errors; CSRF; Open API fallback
Workflow, pages and regression of original rules	16	All pages for all three roles; order conversion deducts stock; completed task creates one training record; insufficient stock refused; deduction and refund; past due date refused; status workflow; artisan on leave cannot be assigned; access control
Fresh installation	1	With no data and no model, the first prediction generates data, trains and answers

Manual verification complemented the tests. Every URL of the system (37 original and new pages, including Django Admin) was opened as admin, manager and viewer, and each role received exactly the access it should, with no server error. The dashboard and all new pages were opened in a real browser, in light and dark mode, with no JavaScript errors. The HTTP path was tested end to end on a running server: API key creation, order streaming, training over HTTP while the server was in use, and rejection of an invalid key. Three incoming orders were converted into tasks through the normal form, and the warehouse stock decreased exactly by the Bill of Materials requirement.

11. Integration with the Existing System

The extension is additive: no original model field, URL or business rule was removed, the original tables were not altered, and the original database is kept unchanged as a backup. The few existing files that had to be touched are listed in Table 14.

Table 14: *Existing files that were modified*

File	Change
config/settings.py, config/urls.py	Registered the analytics app and its settings; added the /analytics/ and /api/v1/ routes
production/views.py	The dashboard view adds the AI context inside a try/except block, so the original dashboard always renders
production/models.py	Bug fix: the past-due-date rule applies only to new tasks or changed due dates
Templates (base, dashboard, task detail)	AI & Analytics sidebar section, live alert bell, new KPI row and AI section, AI Risk Assessment card; HR links and Edit button shown only to staff
requirements.txt	Re-saved as UTF-8; added numpy, scipy, scikit-learn and joblib

Testing the original system under all three roles revealed four small defects, which were corrected. An overdue task could not be saved, so its progress could not be recorded unless its due date was also moved; the rule now applies only when a task is created or its due date is changed. Read-only users saw roster links that silently sent them back to the dashboard, and an Edit Task button; both are now shown only to staff. Finally, the bell icon always showed "!", and now shows the real number of alerts. The delivered database contains the 2,000 synthetic records, the 3 live records, the trained model versions, 18 demonstration orders and their audit log; three low-risk orders were converted into tasks TSK-23, TSK-24 and TSK-25 through the normal workflow, so the materials were deducted by the original stock rules.

12. Security Considerations

- **API keys** are hashed with SHA-256 and shown only once; they can be revoked, and their last use is recorded.
- **Least privilege:** writes need a staff account, viewers are read-only, and the internal service account used for training cannot log in.
- **CSRF protection** is kept for every browser POST, and every API write is validated with field-level errors.

- **Data minimisation and audit:** salaries are not exposed by the API, and every prediction is logged with its inputs, outputs, model version and user.
- **Resilience:** the AI layer is isolated with fail-safe wrappers, so an analytics error can never block a production task or the dashboard.
- **Before a real deployment** (beyond the scope of the course): move the secret key to an environment variable, set DEBUG to False, configure the allowed hosts, enable HTTPS and add rate limiting to the API.

13. Challenges and Lessons Learned

Table 15 lists the main challenges met during development and how each was solved.

Table 15: *Main challenges and their solutions*

Challenge	Problem	Solution
Alert count stayed at zero	The original task model reads its own fields when it loads, so a partial query caused endless recursion	The query was changed and a comment added to explain why
Unrealistic synthetic data	The first synthetic history was too pessimistic, with 59 % of batches late	Slack and workload effects were recalibrated to a realistic 34 %
Stock shortage hidden	A job with a severe shortage could be shown as Low risk	The restock wait now grows with the shortage; such jobs are labelled Blocked until restocked
Data leakage	Artisan history could include the batch being predicted	History uses only earlier batches, and a test checks it
Model portability	A model saved with one scikit-learn version may not load on another computer	Self-healing retraining when the file is missing or incompatible
Internet dependence	The exchange-rate service may be unreachable	Cache, last good value, then a fallback rate

Three lessons stand out. A documented synthetic process is a legitimate way to build and test a machine-learning pipeline when real data is scarce, as long as its limits are stated. A prediction is only useful to a manager when it is explained. And adding intelligence to a working system is safest when the new code is isolated and the old rules are protected by regression tests.

14. Limitations and Future Work

- **Real data and calibration.** Replace the synthetic history progressively with real completed tasks (already automated), retrain periodically, and calibrate the probabilities once enough real data exists.
- **Push notifications.** Replace the 15-second polling of the live feed with WebSockets and send e-mail alerts for high-risk orders.
- **More models.** Add time-series forecasting of orders per product, and a third model that predicts the defect rate of a batch, since the records already hold defective units.
- **API ecosystem.** Generate an OpenAPI (Swagger) specification and add rate limiting and key scopes.
- **Production deployment.** Move to a client-server database such as PostgreSQL and a production web server.

15. Conclusion

The final project turned the Le Mieux Production Management System from a system that records production into one that anticipates it. A versioned REST API now opens the ERP to other systems and feeds a machine-learning pipeline that reads its data through that API. Two models, selected after a fair comparison of three algorithms each and evaluated on data they had never seen, predict whether a job will be late (ROC-AUC 0.934) and how long it will take (average error 1.70 days). They are deployed inside Django with versioning, caching and self-healing, and they score every new incoming order the moment it arrives.

For the production manager, the result is visible on the dashboard: which open tasks are drifting, which new orders are risky and why, which artisan should take a job, and which materials will run short. Every prediction comes with understandable recommendations and can be turned into a real task through the original workflow, whose rules are unchanged and verified by automated tests. Because every completed task feeds back into the training data,

the system will become more accurate the longer the factory uses it. All five requirements set by the lecturer were met without breaking the way the existing system works.

References

AdminLTE. (n.d.). *AdminLTE admin dashboard template*. https://adminlte.io

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

Chart.js. (n.d.). *Chart.js documentation*. https://www.chartjs.org

Django Software Foundation. (n.d.). *Django documentation*. https://docs.djangoproject.com

ExchangeRate-API. (n.d.). *Open access exchange rate API*. https://www.exchangerate-api.com

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine].

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232.

Group 13. (2026). *Production Management System: Midterm project report*. Adventist University of Central Africa, Web Technologies and API Services.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Appendix A: Installation and Operation Guide

Requirements. Python 3.12 or newer, pip and an internet connection for the first installation of the packages, and a modern web browser. The delivered folder contains the source code, the ready-to-use database, the trained models, this report, a change log, screenshots and a backup of the original database.

Installation. Open a terminal in the folder that contains manage.py and run:

```
python -m venv venv
venv\Scripts\activate          (Windows)
source venv/bin/activate       (macOS / Linux)
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Then open http://127.0.0.1:8000/ and log in with one of the demonstration accounts in Table 16. On a different computer, the first AI page may take about 10 seconds because the models retrain themselves once for the local scikit-learn version.

Table 16: *Demonstration login accounts (for the local demonstration only)*

Username	**Password**	**Role**
admin	Admin@2026	Superuser: all features and Django Admin
HR1	Web@tech2	Manager (staff): all CRUD, orders, retraining, API keys
User	Web@tech3	Viewer: read-only pages and API, What-if Predictor

Useful commands. python manage.py test runs the 40 automated tests; setup_ai rebuilds the AI module; train_models train and deploys new model versions from the API; simulate_orders --count 10 --interval 3 streams new orders (add --api-url and --api-key to send them over HTTP); and create_api_key --user NAME creates an API key, printed once.

Appendix B: Glossary

Table 17: *Glossary of terms*

Term	Meaning
API key	A secret string that identifies a program calling the API, with the permissions of its owner
BOM (Bill of Materials)	The list of materials and quantities needed to make one unit of a product
Feature	A measurable input given to a model, such as quantity or workload
Gradient boosting	An algorithm that builds many small decision trees, each correcting the errors of the previous ones
MAE	Mean absolute error: the average size of the prediction error, here in days
ROC-AUC	Probability that the model gives a higher risk to a late job than to an on-time job (0.5 is random, 1 is perfect)
Slack ratio	Days allowed divided by the standard duration; below 1 means an unrealistic plan
Synthetic data	Data generated by a documented process that imitates reality, used when real data is insufficient
Target leakage	Accidentally giving the model information about the answer it must predict
Time-based split	Training on older data and testing on newer data, as in real use